

AI & ASSISTED CODING

LAB-5.3

NAME:N.SATWIK A

BATCH:28

HT NO.:2303A51690

Task 1: Privacy and Data Security in AI-Generated Code

Scenario

AI tools can sometimes generate insecure authentication logic.

Task Description

Use an AI tool to generate a simple login system in Python.

Analyze the generated code to check:

- Whether credentials are hardcoded
- Whether passwords are stored or compared in plain text
- Whether insecure logic is used

Then, revise the code to improve security (e.g., avoid hardcoding, use input validation).

Expected Output

- AI-generated login code
- Identification of security risks
- Revised secure version of the code
- Brief explanation of improvements

Prompt: Generate a simple login system in Python.

Code & Output:

The screenshot shows a Google Colab notebook interface. The browser tab is titled 'AI&AC_5.3.ipynb - Colab'. The address bar shows the URL 'colab.research.google.com/drive/1IFC8b9HU562zWneUMy4nQlg8L8nR7gbZ'. The notebook has a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu bar is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. The notebook content area shows a code cell with the following Python code:

```
[2]
✓ 11s
username = "admin"
password = "admin123"

user = input("Enter username: ")
pwd = input("Enter password: ")

if user == username and pwd == password:
    print("Login successful")
else:
    print("Login failed")
```

The output of the code cell is:

```
... Enter username: admin
Enter password: admin123
Login successful
```

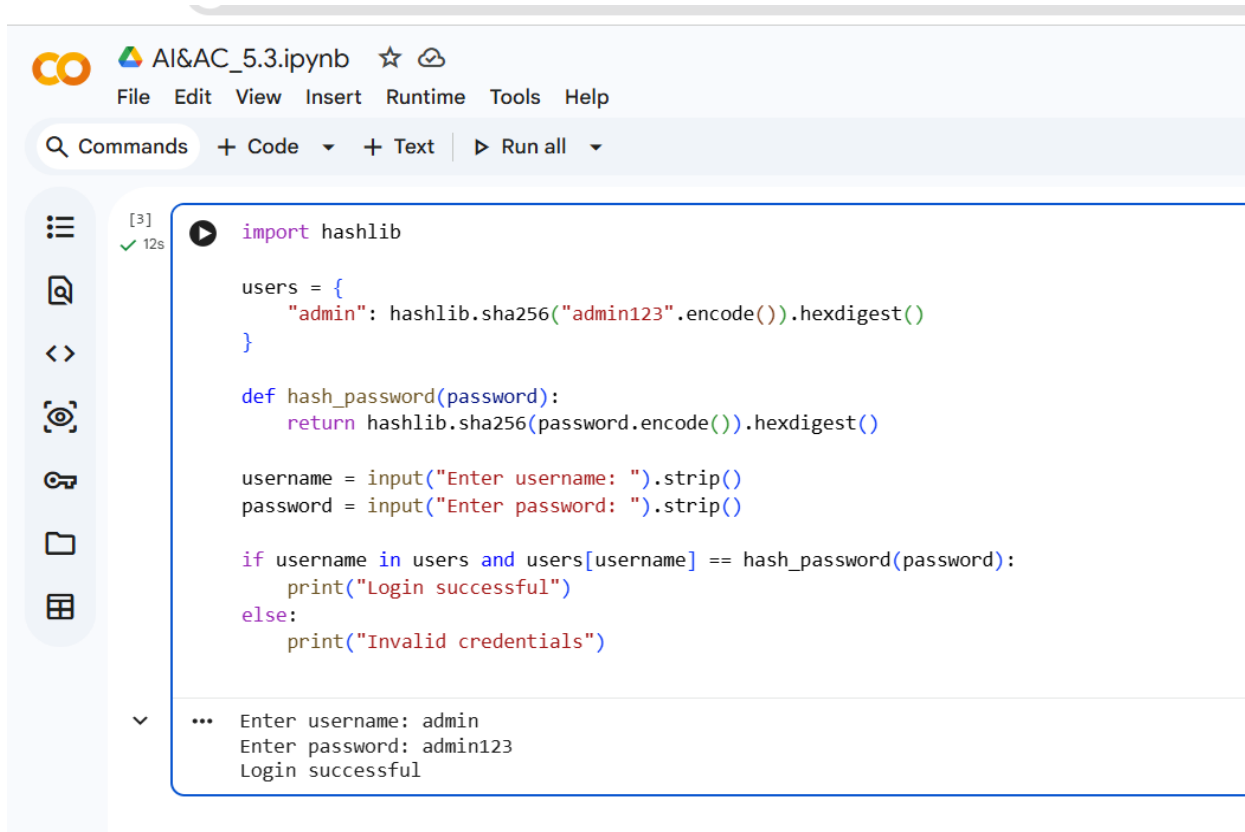
Explanation:

- The program stores username and password directly in the code.
- It takes user input and compares it with stored credentials.
- If both match, login is successful.

➔ Security Issues Identified

- Credentials are hardcoded
- Password stored and compared in plain text
- No input validation
- Vulnerable to data leakage and brute-force attacks

→ Revised secure version of the code



The screenshot shows a Jupyter Notebook titled 'AI&AC_5.3.ipynb'. The interface includes a top menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a search bar and buttons for 'Commands', '+ Code', '+ Text', and 'Run all'. On the left side, there is a sidebar with icons for file management and a list of cells. The main area displays a Python code cell with the following code:

```
[3] ✓ 12s
import hashlib

users = {
    "admin": hashlib.sha256("admin123".encode()).hexdigest()
}

def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

username = input("Enter username: ").strip()
password = input("Enter password: ").strip()

if username in users and users[username] == hash_password(password):
    print("Login successful")
else:
    print("Invalid credentials")
```

Below the code, the output of the script is shown:

```
... Enter username: admin
Enter password: admin123
Login successful
```

Explanation of Improvements:

- Passwords are hashed using SHA-256.
- No plain-text password storage.
- Improved security and privacy protection.

Task 2: Bias Detection in AI-Generated Decision Systems

Scenario

AI systems may unintentionally introduce bias.

Task Description

Use AI prompts such as:

- "Create a loan approval system"
- Vary applicant names and genders in prompts

Analyze whether:

- The logic treats certain genders or names unfairly
- Approval decisions depend on irrelevant personal attributes

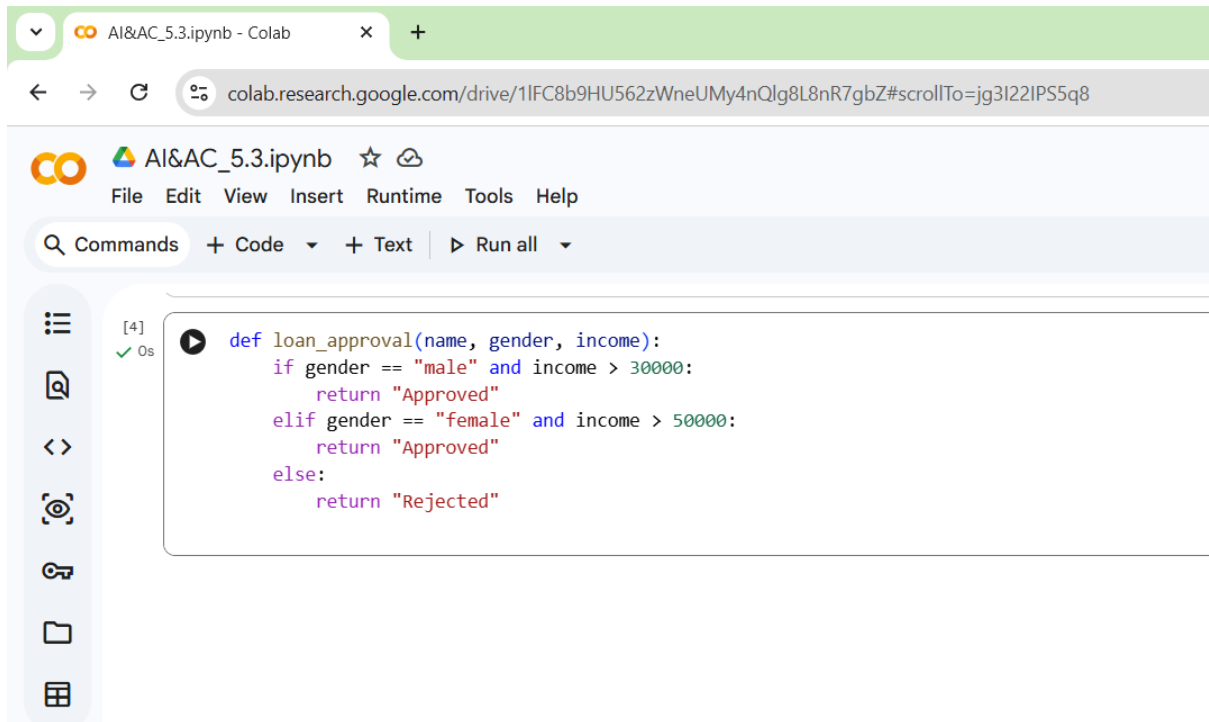
Suggest methods to reduce or remove bias.

Expected Output

- Python code generated by AI
- Identification of biased logic (if any)
- Discussion on fairness issues
- Mitigation strategies

Prompt: Create a loan approval system in Python.

Code & Output:



The screenshot shows a Google Colab notebook interface. The browser address bar displays the URL: `colab.research.google.com/drive/1IFC8b9HU562zWneUMy4nQlg8L8nR7gbZ#scrollTo=jg3l22IPS5q8`. The notebook title is "AI&AC_5.3.ipynb". The menu bar includes "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". Below the menu is a toolbar with "Commands", "+ Code", "+ Text", and "Run all". On the left sidebar, there are icons for file management and execution. The main code cell, labeled "[4]" and "0s", contains the following Python code:

```
def loan_approval(name, gender, income):  
    if gender == "male" and income > 30000:  
        return "Approved"  
    elif gender == "female" and income > 50000:  
        return "Approved"  
    else:  
        return "Rejected"
```

Explanation:

- Loan approval is based on income and gender.
- Different income thresholds are used for males and females.

Bias Analysis:

- Gender-based discrimination
- Female applicants require higher income
- Gender is an irrelevant factor for loan approval.

Bias-Free Revised Code:-

```
s  def loan_approval(income, credit_score):  
    if income >= 40000 and credit_score >= 650:  
        return "Approved"  
    else:  
        return "Rejected"
```

Bias Mitigation Explanation

- Removed gender and name.
- Approval depends only on financial criteria.
- Ensures fairness and equality.

Task3: Transparency and Explainability in AI-Generated Code (Recursive Binary Search)

Scenario

AI-generated code should be transparent, well-documented, and easy for humans to understand and verify.

Task Description

Use an AI tool to generate a Python program that:

- Implements Binary Search using recursion
- Searches for a given element in a sorted list
- Includes:
 - o Clear inline comments
 - o A step-by-step explanation of the recursive logic

After generating the code, analyze:

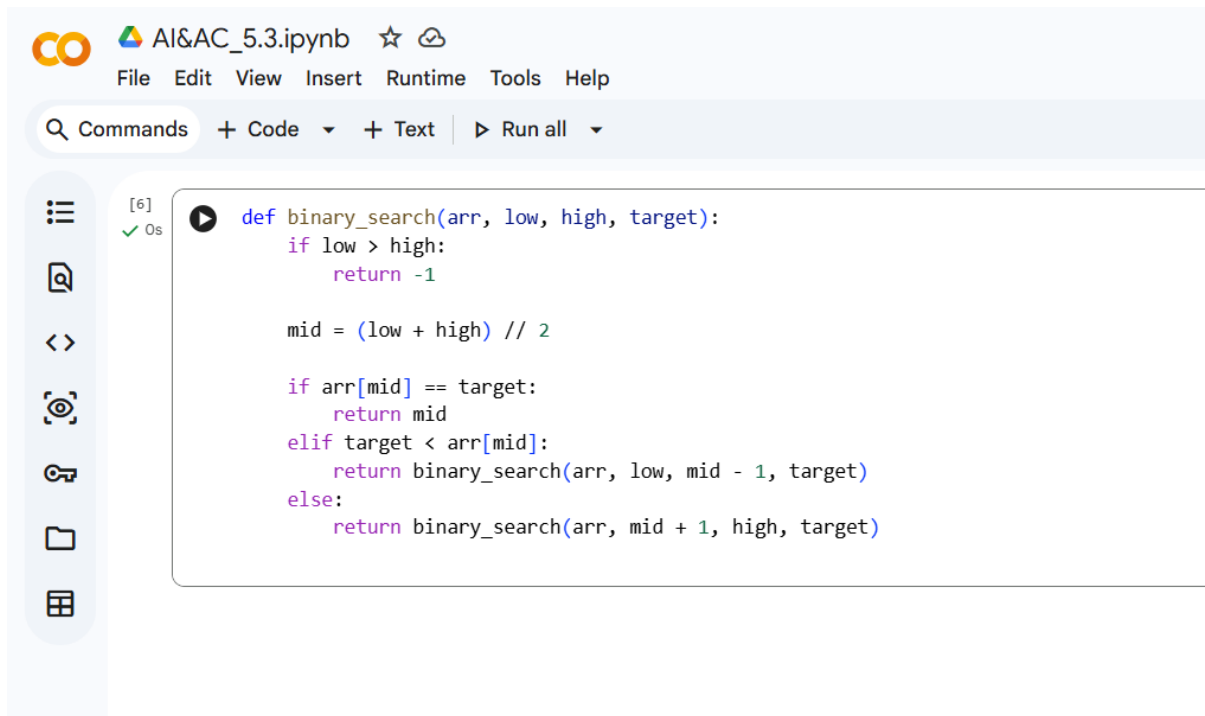
- Whether the explanation clearly describes the base case and recursive case
- Whether the comments correctly match the code logic
- Whether the code is understandable for beginner-level students

Expected Output

- Python program for recursive binary search
- AI-generated comments and explanation
- Student's assessment on clarity, correctness, and transparency

Prompt: Write a Python program for recursive binary search with explanation.

Code & Output:

A screenshot of a Jupyter Notebook interface. The top bar shows the file name 'AI&AC_5.3.ipynb' and a star icon. Below it is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. A search bar labeled 'Commands' is followed by buttons for '+ Code', '+ Text', and 'Run all'. On the left side, there is a vertical toolbar with icons for a list, a magnifying glass, a double arrow, a camera, a key, a folder, and a grid. The main area displays a code cell with a play button icon and a status indicator '[6] ✓ 0s'. The code is a recursive function for binary search.

```
def binary_search(arr, low, high, target):
    if low > high:
        return -1

    mid = (low + high) // 2

    if arr[mid] == target:
        return mid
    elif target < arr[mid]:
        return binary_search(arr, low, mid - 1, target)
    else:
        return binary_search(arr, mid + 1, high, target)
```

Explanation:

- Base case: when $low > high$, element not found.
- case: search left or right half based on comparison.
- Efficient search with time complexity $O(\log n)$.

Transparency Analysis:

- Base case clearly defined
- Recursive calls are understandable
- Beginner-friendly logic and comments

Task 4: Ethical Evaluation of AI-Based Scoring Systems

Scenario

AI-generated scoring systems can influence hiring decisions.

Task Description

Ask an AI tool to generate a job applicant scoring system based on features such as:

- Skills
- Experience
- Education

Analyze the generated code to check:

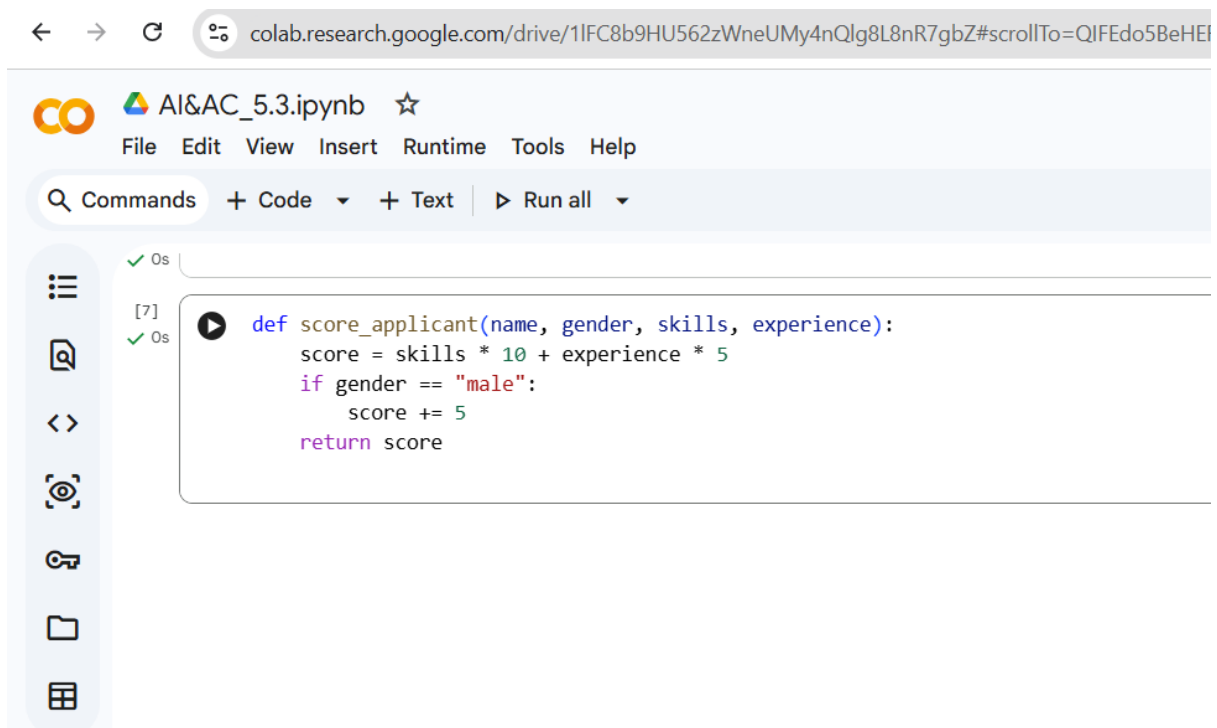
- Whether gender, name, or unrelated features influence scoring
- Whether the logic is fair and objective

Expected Output

- Python scoring system code
- Identification of potential bias (if any)
- Ethical analysis of the scoring logic

Prompt: Create a job applicant scoring system in Python.

Code and Output:



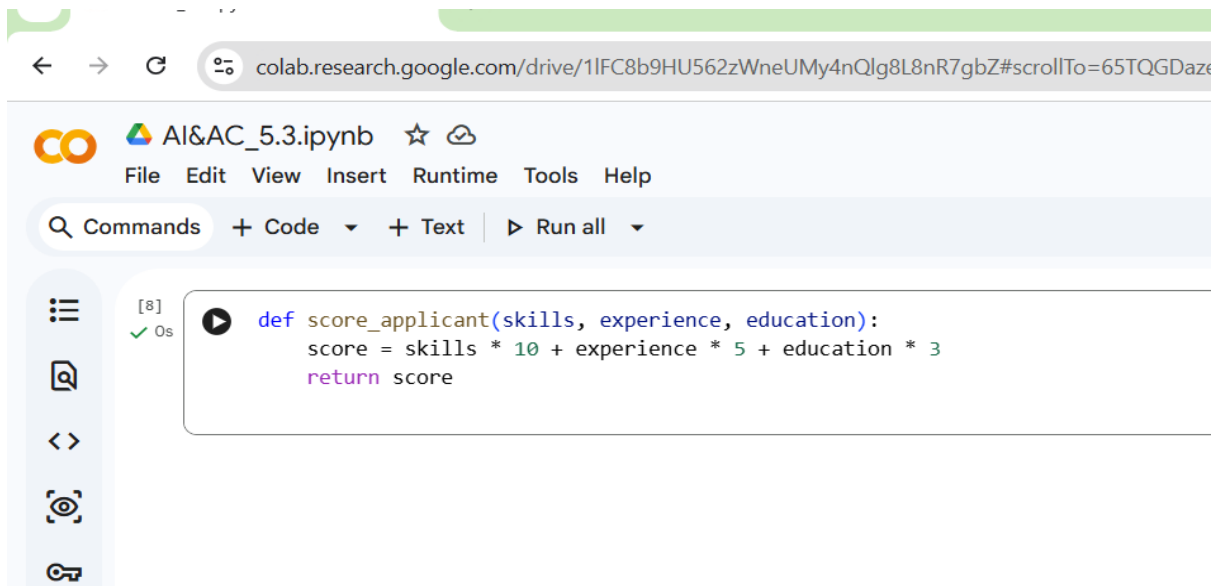
The screenshot shows a Google Colab notebook interface. The browser address bar at the top displays the URL: `colab.research.google.com/drive/1IFC8b9HU562zWneUMy4nQlg8L8nR7gbZ#scrollTo=QIFEdo5BeHEI`. The notebook title is `AI&AC_5.3.ipynb`. The menu bar includes `File`, `Edit`, `View`, `Insert`, `Runtime`, `Tools`, and `Help`. Below the menu, there is a toolbar with `Commands`, `+ Code`, `+ Text`, and `Run all`. On the left sidebar, there are icons for file management and execution. The main code cell contains the following Python function:

```
def score_applicant(name, gender, skills, experience):  
    score = skills * 10 + experience * 5  
    if gender == "male":  
        score += 5  
    return score
```

Explanation:

→ Applicant score depends on skills, experience, and gender.

→ **Ethical Revised Code**



Ethical Improvement Explanation:

- Removed gender and name.
- Used job-related features only.
- Promotes objective and fair hiring.
- Male applicants receive extra points.

Ethical Issues Identified:

- Gender bias
- Unfair hiring advantage
- Violates ethical hiring standards

Task 5: Inclusiveness and Ethical Variable Design

Scenario

Inclusive coding practices avoid assumptions related to gender, identity, or roles and promote fairness in software design.

Task Description

Use an AI tool to generate a Python code snippet that processes user or employee details.

Analyze the code to identify:

- Gender-specific variables (e.g., male, female)
- Assumptions based on gender or identity

- Non-inclusive naming or logic

Modify or regenerate the code to:

- Use gender-neutral variable names
- Avoid gender-based conditions unless strictly required
- Ensure inclusive and respectful coding practices

Expected Output

- Original AI-generated code snippet
- Revised inclusive and gender-neutral code
- Brief explanation of:
 - o What was non-inclusive
 - o How inclusiveness was improved

Prompt: Write a Python program to process employee details.

Code & Output:



The screenshot shows a Google Colab notebook interface. The browser address bar displays the URL: `colab.research.google.com/drive/1IFC8b9HU562zWneUMy4nQlg8L8nR7gbZ#scrollTo=LmPxVwFwf8Ro`. The notebook title is "AI&AC_5.3.ipynb". The menu bar includes "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". Below the menu, there are tabs for "Commands", "Code", and "Text", along with a "Run all" button. The code editor shows a Python function:

```
def assign_role(name, gender):
    if gender == "male":
        print(name, "assigned to technical role")
    else:
        print(name, "assigned to support role")
```

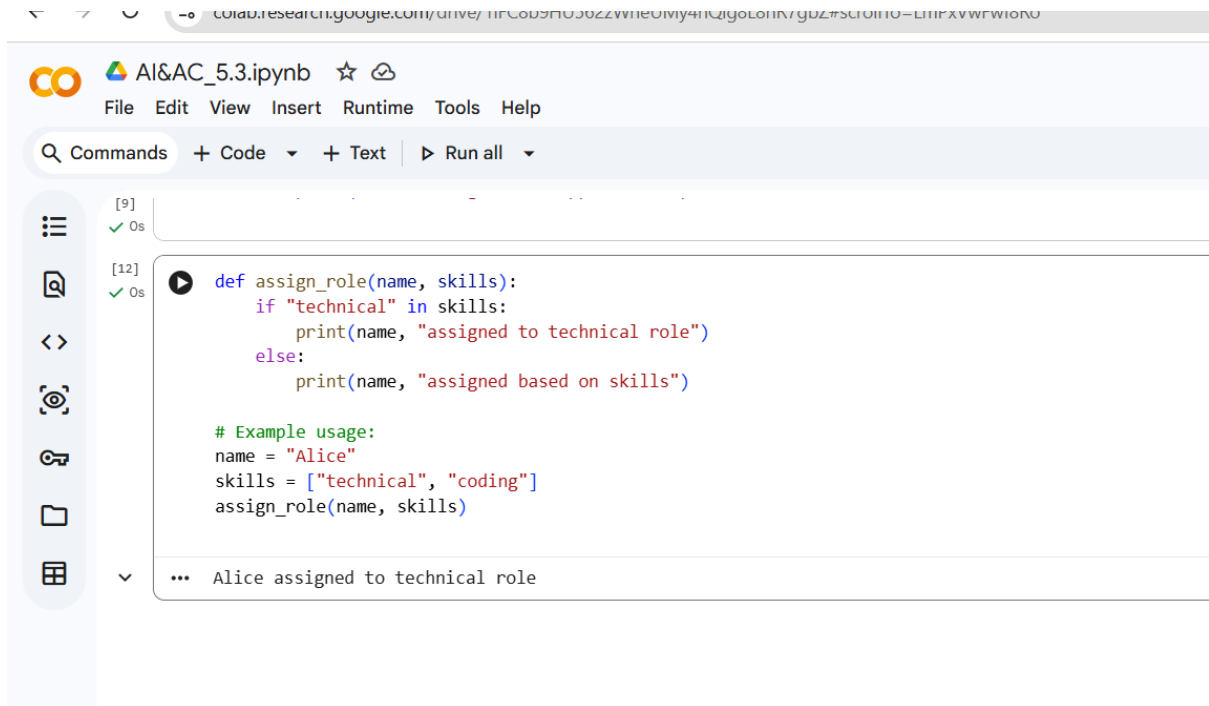
Explanation:

- Role assignment is based on gender.
- Assumes technical roles are for males.

Inclusiveness Issues:

- Gender stereotypes
- Non-inclusive logic
- Unethical role assignment

Inclusive Revised Code:



```
[9] ✓ Os  
[12] ✓ Os  
def assign_role(name, skills):  
    if "technical" in skills:  
        print(name, "assigned to technical role")  
    else:  
        print(name, "assigned based on skills")  
  
# Example usage:  
name = "Alice"  
skills = ["technical", "coding"]  
assign_role(name, skills)  
  
... Alice assigned to technical role
```

Inclusiveness Improvement Explanation:

- Removed gender-based assumptions.
- Role assignment depends on skills.
- Encourages inclusive and ethical coding.